TEMPLE COLLEGE
WORKFORCE DEVELOPMENT

CHIEFOS

STUDENT REVIEW PACKET

Week 17

CORS + fetch, create-item form, extend with Continue — Module 06 checkpoint

Phase 3 — AI-Assisted Development · Weeks 13–18 · Modules 05–06 + Capstone

No D2L login required. This is the same lesson text posted in Brightspace for this week. Work the lessons in order on your own machine. Keep each checkpoint screenshot or pasted output. Bring it to the live session, or email it to your instructor, instead of submitting in D2L. Module quizzes stay in D2L and are not included here.

In this packet

1. 06.04 CORS + Fetch Items
2. 06.05 Create-Item Form
3. 06.06 Extend with Continue.dev
4. 06.07 Final Checkpoint

Learning objective: Add `CORSMiddleware` to the FastAPI backend and fetch the items list from the browser, rendering each item as a HeroUI Card with loading, error, and empty states.

Lesson 04: CORS + Fetch Items

By the end, your frontend will fetch the items list from FastAPI and render each item as a HeroUI Card. You'll have hit the CORS wall and added the middleware that breaks through it.

~25 minutes of typing

> Browsers block JavaScript on `http://localhost:3000` from calling `http://localhost:8000` by default — that's **CORS** (Cross-Origin Resource Sharing). FastAPI has middleware to allow it. Today you wire it up.

Step 1: feel the CORS wall first

1. Make sure the FastAPI backend is running:

```
cd ~/docker-course/fastapi-app
docker compose up -d
sleep 2
curl -s http://localhost:8000/items | head -c 60
```

You should see JSON.

2. Make sure a few items exist (so we have something to render):

```
curl -s -X POST http://localhost:8000/items -H "Content-Type: application/json" -d
'{"name":"Apple","price":0.5}' > /dev/null
curl -s -X POST http://localhost:8000/items -H "Content-Type: application/json" -d
'{"name":"Bread","price":2.5,"is_offer":true}' > /dev/null
curl -s http://localhost:8000/items | python3 -m json.tool
```

3. Move to the frontend project.

```
cd ~/docker-course/fastapi-frontend
```

4. Replace `app/page.tsx` with a fetch attempt:

```tsx
"use client";

import { Card, CardBody, CardHeader } from "@heroui/react";
import { useEffect, useState } from "react";

type Item = {
  id: number;
  name: string;
  price: number;
  is_offer: boolean | null;
};

export default function Home() {
  const [items, setItems] = useState<Item[]>([]);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    fetch("http://localhost:8000/items")
      .then((res) => res.json())
      .then((data) => setItems(data))
      .catch((err) => setError(err.message));
  }, []);

  return (
    <main className="flex min-h-screen flex-col items-center gap-4 p-8">
      <h1 className="text-3xl font-bold">Items</h1>
      {error && <p className="text-red-500">Error: {error}</p>}
      <div className="flex flex-col gap-2 w-96">
        {items.map((item) => (
          <Card key={item.id}>
            <CardHeader className="flex justify-between">
              <span className="font-semibold">{item.name}</span>
              <span>${item.price.toFixed(2)}</span>
            </CardHeader>
            <CardBody>
              {item.is_offer ? (
                <span className="text-green-600">On offer</span>
              ) : (
                <span className="text-gray-500">Regular price</span>
              )}
            </CardBody>
          </Card>
        ))}
      </div>
```

```
    </main>
  );
}
```

5. Save. Open `http://localhost:3000` in the browser.

6. The page is empty (no items rendered). Open the browser dev tools (F12 or right-click → Inspect) and look at the **Console** tab.

7. You should see something like:

```
Access to fetch at 'http://localhost:8000/items' from origin
'http://localhost:3000' has been blocked by CORS policy: ...
```

That's the wall. Now we break through it.

Step 2: add CORS middleware to FastAPI

8. Open the **backend** project — `~/docker-course/fastapi-app/main.py` — in VS Code.

9. Add the import at the top with the other FastAPI imports:

```
from fastapi.middleware.cors import CORSMiddleware
```

10. Below `app = FastAPI()`, add the middleware:

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

11. Save. Watch the api logs — uvicorn should hot-reload (Module 03 lesson 09 wired bind-mount + `--reload` for exactly this moment).

```
docker compose -f ~/docker-course/fastapi-app/docker-compose.yml logs api --tail=5
```

You should see `Application startup complete`.

12. Refresh `http://localhost:3000` in the browser. **The items render.**

13. Open the browser console — no more CORS errors.

Step 3: read the four key arguments

- **`allow_origins=["http://localhost:3000"]`** — exactly which origin can call your API. Use a list. Don't use `["*"]` in production with credentials.
- **`allow_credentials=True`** — allows cookies/auth headers to ride along.
- **`allow_methods=["*"]`** — allow all HTTP methods (GET, POST, DELETE, etc.).
- **`allow_headers=["*"]`** — allow all request headers, including custom ones like `X-Token` from Module 03.

Step 4: prove the round-trip works

14. Add a fresh item from the terminal:

```
curl -s -X POST http://localhost:8000/items \
  -H "Content-Type: application/json" \
  -d '{"name":"Milk","price":3.0}' > /dev/null
```

15. Refresh the browser. Milk shows up in the list.

16. Look at the browser **Network** tab in dev tools. The request to `http://localhost:8000/items` should have status 200 and the response body should be your JSON.

Step 5: handle loading and empty states

17. Update `app/page.tsx` to show a loading state and an "empty" state:

```tsx
"use client";

import { Card, CardBody, CardHeader, Spinner } from "@heroui/react";
import { useEffect, useState } from "react";

type Item = {
  id: number;
  name: string;
  price: number;
  is_offer: boolean | null;
};

export default function Home() {
  const [items, setItems] = useState<Item[]>([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    fetch("http://localhost:8000/items")
      .then((res) => res.json())
      .then((data) => setItems(data))
      .catch((err) => setError(err.message))
      .finally(() => setLoading(false));
  }, []);

  return (
    <main className="flex min-h-screen flex-col items-center gap-4 p-8">
      <h1 className="text-3xl font-bold">Items</h1>

      {loading && <Spinner label="Loading items..." />}
      {error && <p className="text-red-500">Error: {error}</p>}
      {!loading && !error && items.length === 0 && (
        <p className="text-gray-500">No items yet — add one!</p>
      )}

      <div className="flex flex-col gap-2 w-96">
        {items.map((item) => (
          <Card key={item.id}>
            <CardHeader className="flex justify-between">
              <span className="font-semibold">{item.name}</span>
              <span>${item.price.toFixed(2)}</span>
            </CardHeader>
            <CardBody>
              {item.is_offer ? (
```

```
                <span className="text-green-600">On offer</span>
              ) : (
                <span className="text-gray-500">Regular price</span>
              )}
            </CardBody>
          </Card>
        ))}
      </div>
    </main>
  );
}
```

18. Save. Refresh the browser — you'll briefly see a spinner before the items load.

19. Stop the FastAPI backend to feel the error path:

```
docker compose -f ~/docker-course/fastapi-app/docker-compose.yml stop api
```

20. Refresh the browser. After a moment, you should see the red error text.

21. Restart it:

```
docker compose -f ~/docker-course/fastapi-app/docker-compose.yml start api
```

22. Refresh — items return.

Step 6: commit

23. Backend changes:

```
cd ~/docker-course/fastapi-app
git add main.py
git commit -m "Add CORS middleware for frontend on :3000"
```

24. Frontend changes:

```
cd ~/docker-course/fastapi-frontend
git add app/page.tsx
git commit -m "Fetch and render items from FastAPI"
```

25. Log it.

```
echo "Frontend fetches /items from FastAPI; CORS middleware added" >> ~/docker-course/notes/notes.txt
```

Watch out

- **CORS errors only happen in browsers.** `curl` and `fetch` from the same origin (or from server-side code) don't trigger CORS. So `curl http://localhost:8000/items` always works — it's only the browser that complains.
- **`allow_origins=["*"]` + `allow_credentials=True` is invalid.** The spec forbids that combination. If you want both, list specific origins.
- **The `/api/...` proxy pattern.** Some Next.js apps avoid CORS by routing `/api/*` to the backend through the Next.js server. We're using direct fetch with CORS for clarity. Either is fine.
- **Configure `allow_origins` per environment.** Hardcoding `http://localhost:3000` is fine for dev. For staging/prod, use env vars. We'll cross that bridge when we deploy.

Checkpoint

```
grep "CORSMiddleware" ~/docker-course/fastapi-app/main.py
```

Should match.

In the browser at `http://localhost:3000`:

- An "Items" heading
- A loading spinner briefly, then your items as Cards
- No CORS errors in the console

```
curl -s -I -X OPTIONS http://localhost:8000/items \
  -H "Origin: http://localhost:3000" \
  -H "Access-Control-Request-Method: GET" | grep -i "access-control"
```

Should print headers like `access-control-allow-origin: http://localhost:3000`.

Next up: 05-create-item-form.md — Add a form to create items

Lab — do it for real. Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.

Learning objective: Build a HeroUI form that POSTs new items to FastAPI, converts input strings to the types the API expects, surfaces server validation errors, and reloads the list inline on success.

Lesson 05: Create-Item Form

By the end, you will have a HeroUI form that POSTs new items to FastAPI and refreshes the list inline — full create-and-display loop.

~20 minutes of typing

---

1. In `~/docker-course/fastapi-frontend/app/page.tsx`, you currently have a list view. We're adding a form above it that creates new items.

2. Replace the contents of `app/page.tsx` with:

```
"use client";

import {
  Button,
  Card,
  CardBody,
  CardHeader,
  Checkbox,
  Input,
  Spinner,
} from "@heroui/react";
import { FormEvent, useEffect, useState } from "react";

type Item = {
  id: number;
  name: string;
  price: number;
  is_offer: boolean | null;
};

const API = "http://localhost:8000";

export default function Home() {
  const [items, setItems] = useState<Item[]>([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);

  // form state
  const [name, setName] = useState("");
  const [price, setPrice] = useState("");
  const [isOffer, setIsOffer] = useState(false);
  const [submitting, setSubmitting] = useState(false);
  const [formError, setFormError] = useState<string | null>(null);

  async function loadItems() {
    setLoading(true);
    setError(null);
    try {
      const res = await fetch(`${API}/items`);
      if (!res.ok) throw new Error(`HTTP ${res.status}`);
      setItems(await res.json());
    } catch (err) {
      setError(err instanceof Error ? err.message : String(err));
    } finally {
```

```
      setLoading(false);
    }
  }

  useEffect(() => {
    loadItems();
  }, []);

  async function handleSubmit(e: FormEvent) {
    e.preventDefault();
    setSubmitting(true);
    setFormError(null);
    try {
      const res = await fetch(`${API}/items`, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({
          name,
          price: Number(price),
          is_offer: isOffer,
        }),
      });
      if (!res.ok) {
        const detail = await res.text();
        throw new Error(`HTTP ${res.status}: ${detail}`);
      }
      // success — clear form and reload list
      setName("");
      setPrice("");
      setIsOffer(false);
      await loadItems();
    } catch (err) {
      setFormError(err instanceof Error ? err.message : String(err));
    } finally {
      setSubmitting(false);
    }
  }

  return (
    <main className="flex min-h-screen flex-col items-center gap-6 p-8">
      <h1 className="text-3xl font-bold">Items</h1>

      <Card className="w-96">
        <CardHeader>
          <h2 className="text-lg font-semibold">Add an item</h2>
```

```
        </CardHeader>
        <CardBody>
          <form onSubmit={handleSubmit} className="flex flex-col gap-3">
            <Input
              label="Name"
              value={name}
              onValueChange={setName}
              isRequired
            />
            <Input
              label="Price"
              type="number"
              step="0.01"
              value={price}
              onValueChange={setPrice}
              isRequired
            />
            <Checkbox isSelected={isOffer} onValueChange={setIsOffer}>
              On offer
            </Checkbox>
            <Button
              type="submit"
              color="primary"
              isLoading={submitting}
              isDisabled={!name || !price}
            >
              Add item
            </Button>
            {formError && (
              <p className="text-sm text-red-500">{formError}</p>
            )}
          </form>
        </CardBody>
      </Card>

      <div className="flex flex-col gap-2 w-96">
        {loading && <Spinner label="Loading items..." />}
        {error && <p className="text-red-500">Error: {error}</p>}
        {!loading && !error && items.length === 0 && (
          <p className="text-gray-500">No items yet — add one!</p>
        )}
        {items.map((item) => (
          <Card key={item.id}>
            <CardHeader className="flex justify-between">
              <span className="font-semibold">{item.name}</span>
```

```
              <span>${item.price.toFixed(2)}</span>
            </CardHeader>
            <CardBody>
              {item.is_offer ? (
                <span className="text-green-600">On offer</span>
              ) : (
                <span className="text-gray-500">Regular price</span>
              )}
            </CardBody>
          </Card>
        ))}
      </div>
    </main>
  );
}
```

3. Save. Watch the dev-server terminal hot-reload.

4. In the browser at `http://localhost:3000`, you should see the form Card above the items list.

Drill the form

5. Type `Cheese` in Name, `5.00` in Price. Click **Add item**.

6. The button shows a loading spinner briefly (HeroUI's `isLoading` prop), then the form clears, then the list reloads with `Cheese` at the bottom.

7. Confirm it's really in the database:

```
curl -s http://localhost:8000/items | python3 -m json.tool | tail -n 10
```

Cheese should be there.

8. And in psql:

```
docker compose -f ~/docker-course/fastapi-app/docker-compose.yml exec db \
  psql -U postgres -d appdb -c "SELECT * FROM items WHERE name='Cheese';"
```

Drill validation errors

9. Try adding an item with `price = -1` (the FastAPI lesson 11 validation should reject it).

10. Click **Add item**. The Add button works (no client-side number-range check in our code), but the response from the server is a 400:

```
HTTP 400: {"detail":"price must be positive"}
```

That message appears in red below the Add button — exactly the behavior the form wires up.

11. Try a non-numeric value — type `free` in Price. Note the Input's `type="number"` mostly prevents this in modern browsers; on older ones the form-level error path catches it.

12. Try the `is_offer` checkbox. Add an item with it checked — the list shows it as **On offer** in green.

Toggle the form into a Modal (optional polish)

Skip this section if pressed for time.

13. Real apps often put creation forms in a modal. HeroUI has a Modal component. Highlight the entire form Card and use Continue.dev:

- **Ctrl+I**, prompt: ``` `Wrap this form in a HeroUI Modal that opens when an "Add item" button is clicked. Use the useDisclosure hook from @heroui/react. Keep the form contents identical.` ```
- Read the diff. Accept if reasonable.

14. The page now shows a small "Add item" button at the top; clicking it opens a modal with the form.

Commit

15. Stage and commit:

```
cd ~/docker-course/fastapi-frontend
git add app/page.tsx
git commit -m "Add create-item form (Card + Input + Checkbox + Button)"
```

16. Log it.

```
echo "Frontend create-item form: working end-to-end" >> ~/docker-
course/notes/notes.txt
```

Watch out

- **`onValueChange`, not `onChange`.** HeroUI's Input/Checkbox/Switch use `onValueChange` which gives you the value directly. `onChange` would give you an Event, which works but isn't idiomatic for HeroUI.
- **`Number(price)` not `price`.** Input values are strings. The API expects a number for `price`. Convert before sending.
- **Always `await loadItems()` after a successful POST.** Otherwise the new item shows up only on next refresh.
- **Don't put the API URL in 12 places.** The constant `const API = "http://localhost:8000"` is the seed of an env var (`NEXT_PUBLIC_API_URL`). We'll skip env vars today; flag it for refactor.

Checkpoint

```
grep "handleSubmit" app/page.tsx
```

Should match.

In the browser:

- The form accepts name + price + isOffer.
- The Add button is disabled until both name and price have values.
- Submitting creates an item that immediately appears in the list below.
- Submitting an invalid price shows the error message in red.

```
curl -s http://localhost:8000/items | python3 -c "import sys,json;
print(len(json.load(sys.stdin)))"
```

Should print a number that increases each time you submit a valid item.

Next up: 06-extend-with-continue.md — Use Continue.dev's agent mode to add a feature

Lab — do it for real. Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.

Learning objective: Use Continue.dev agent mode on a git feature branch to add a Delete button with a confirmation modal, reviewing every proposed diff and recognizing when to abort a failing agent loop.

Lesson 06: Extend with Continue.dev

By the end, you will have used Continue.dev's agent mode to add a Delete button to each item Card, with a confirmation step — both frontend and backend changes.

~20 minutes

> Module 05 drilled the agentic loop on the FastAPI backend. Today: same loop on the frontend, plus a small backend tweak. Two repos in play, same model, same workflow.

---

1. Make sure both stacks are up.

```
docker compose -f ~/docker-course/fastapi-app/docker-compose.yml ps
```

Both `api` and `db` should be Up.

```
ps aux | grep -v grep | grep "next dev" | head -n 1
```

Should show your `next dev` process.

If either is down, bring it up.

Step 1: branch the frontend

2. Create a feature branch in the frontend repo.

```
cd ~/docker-course/fastapi-frontend
git switch -c add-delete-button
```

Step 2: ask Continue.dev for the change

3. Open the Continue side bar. Switch to **Agent** mode.

4. In VS Code, open `app/page.tsx` so it's the visible file.

5. In Continue, type:

```
@file:app/page.tsx

I want to add a Delete button to each item Card. When clicked, it should:
1. Show a HeroUI confirmation Modal asking "Delete <item name>?"
2. On confirm, DELETE http://localhost:8000/items/<id>
3. On success, reload the items list
4. On error, show an inline error message

Show me the changes before applying.
```

Send.

6. The agent should propose changes — likely involving:

- Adding HeroUI's `Modal`, `ModalContent`, `ModalHeader`, `ModalBody`, `ModalFooter` to imports.
- Adding a `useDisclosure` for the modal.
- Adding state for the item being deleted.
- Adding a delete handler.
- Adding the Button to each Card with an `onPress`.

7. Read every diff. Confirm the approach before approving.

8. Approve the proposed changes one at a time.

Step 3: verify it works

9. Save (the agent should have already saved). Refresh the browser.

10. Each item Card should now have a Delete button.

11. Click Delete on an item. A modal should appear asking for confirmation.

12. Click Confirm. The modal closes; the list reloads; the item is gone.

13. Confirm in the database:

```
docker compose -f ~/docker-course/fastapi-app/docker-compose.yml exec db \
  psql -U postgres -d appdb -c "SELECT count(*) FROM items;"
```

Count went down by 1.

Step 4: feel a real failure mode

14. Now ask the agent for a tricky thing. In Continue (still in agent mode), type:

```
Add an "Undo" button that appears for 5 seconds after a delete, allowing the user
to restore the deleted item. Use a Toast pattern.
```

15. This is harder. The model might:

- Try to use a `Toast` component that doesn't exist in HeroUI's exports.
- Try to call a backend route that doesn't exist (we don't have an "undelete" endpoint).
- Loop trying to make it work.

16. Watch carefully. If the agent goes off the rails:

- **Reject** the broken proposal.
- In the chat, push back: `HeroUI doesn't have a Toast component. Use a Snackbar or an inline-alert pattern instead. And there's no undelete endpoint — for now, re-POST the item using the data we have on the client.`
- Let it try again.

17. If it gets stuck in a loop, abandon the approach: stop the agent, revert any half-made changes (`git checkout app/page.tsx`), and move on. **Knowing when to give up on the agent is a skill.**

Step 5: commit

18. If the Delete button works (skip the Undo bit if it didn't pan out), commit.

```
git add app/page.tsx
git commit -m "Add Delete button with confirmation modal"
```

19. Push the branch.

```
# Assumes you set up a frontend repo on GitHub earlier; if not, skip the push for
now
git remote -v
```

If you have an `origin` set, `git push -u origin add-delete-button` and open a PR on GitHub (Module 04 lesson 10 workflow). If not, just merge locally:

```
git switch main
git merge add-delete-button
git branch -d add-delete-button
```

20. Log it.

```
echo "Continue.dev agent: added Delete button + confirmation modal" >> ~/docker-
course/notes/notes.txt
```

Watch out

- **Read every diff.** The 7B model is good but not infallible — especially on a UI library it may not have seen extensively in training. It will sometimes import components that don't exist or use props that aren't in the current HeroUI version.
- **Both stacks must be running.** If the agent calls a backend endpoint and gets a connection-refused, it might "fix" the wrong thing. Make sure FastAPI is up before agent mode.
- **Branch first, agent second.** Always `git switch -c agent-<task>` before letting the agent loose. Easy revert if the result is junk.
- **Knowing when to abort.** A small model in a loop is a small model in a loop. Three failed attempts on the same task → abort, simplify the request, or do it manually with `Ctrl+I`.

Checkpoint

In the browser:

- Each item Card has a Delete button.
- Clicking Delete opens a confirmation modal.
- Confirming actually removes the item from the list and the database.

```
grep -E "Delete|onPress.*delete" app/page.tsx | head -n 3
```

Should show a Delete button or handler.

```
git log --oneline -1
```

Should show your "Add Delete button" commit.

Next up: 07-final-checkpoint.md — Ship a feature end-to-end

> **Lab — do it for real.** Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.

Learning objective: Ship a client-side, case-insensitive search filter for the items page end-to-end using Continue.dev, verify it in the browser, and commit and merge the change.

Lesson 07: Final Checkpoint

Time: ~20 min — **no instructor help.**

Cumulative drill: ship one new frontend feature end-to-end using Continue.dev as your AI assistant.

The task

Add a **search box** at the top of the items page that filters the displayed items by name as you type. **Client-side filter only** — no new backend route needed (this is purely a frontend exercise).

Requirements

- A HeroUI `Input` with a search icon (or `placeholder="Search items..."`).
- As the user types, the items list filters in real time. Case-insensitive match against `name`.
- Empty search box shows all items.
- The "No items yet" empty state is replaced with "No items match your search" when filtering returns zero results but `items.length > 0`.

You decide how to use Continue: chat for ideas, inline edit for surgical changes, or agent mode for a multi-step change.

Suggested workflow

1. Branch.

```
cd ~/docker-course/fastapi-frontend
git switch main
git switch -c add-item-search
```

2. Use Continue.dev to make the change. Read every diff before accepting.

3. Verify in the browser:

- Type something — items filter live.
- Clear the box — all items return.

- Type something with no matches — see the "No items match" message.

4. Commit, push, open a PR (or merge locally if no GitHub origin).

```
git add app/page.tsx
git commit -m "Add client-side search filter for items"
# git push -u origin add-item-search   # if you set up the frontend on GitHub
git switch main
git merge add-item-search
git branch -d add-item-search
```

Acceptance test

After merging:

In the browser at `http://localhost:3000`:

- A search Input appears above the items list.
- Typing `a` filters to items whose name contains `a` (case-insensitive).
- Clearing the search box restores the full list.
- Typing `zzzzzz` shows the "No items match your search" message.

```
grep -i "search\|filter" app/page.tsx | head -n 5
```

Should show search/filter logic.

```
git log --oneline -1
```

Should show the merge or commit from this checkpoint.

What to submit

A screenshot showing **all of these** in one image:

1. The browser at `localhost:3000` with the search box visible.
2. The list filtered by a typed query.
3. The Continue chat panel showing the prompt you used to build it.
4. A terminal with `git log --oneline -3`.

Self-grade

You pass this module if:

- [ ] You used **Continue.dev with the shared vLLM endpoint (Llama)** to build the feature (chat, edit, or agent mode — any combination).
- [ ] The search box behaves as specified.
- [ ] The change was committed (and pushed/merged if you set up the frontend on GitHub).
- [ ] You relied on the **free** Continue.dev + shared vLLM stack — no paid cloud AI.

```
echo "Module 06 final checkpoint: passed $(date -Iseconds)" >> ~/docker-course/notes/notes.txt
```

What you have now

A real full-stack slice:

- **Backend:** FastAPI + Postgres + SQLAlchemy in Docker Compose.
- **Frontend:** Next.js + HeroUI + Tailwind talking to the backend over CORS.
- **Tooling:** VS Code + Continue.dev + the shared vLLM endpoint (Llama) for AI assistance, GitHub for version control.
- **Workflow:** branch → AI-assisted edit → diff review → commit → PR → merge.

That's the shape of every real product team's setup, just smaller. Don't delete any of this — Modules 07 and 08 build on it.

What's next

Module 07 (sessions 13–14) introduces **agents and tool use** at the API level — building agents that call tools (including your FastAPI endpoints) using the same shared vLLM endpoint (Llama).

Lab — do it for real. Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.